

Generador de señales cuadradas de 100 kHz, 500 kHz, 1 MHz y 2 MHz en lenguaje ensamblador con el microcontrolador ATmega328P

Torres Razo Vicente Alfonso

Universidad Nacional Autónoma de México, Facultad de Estudios Superiores Cuautitlán

October 1, 2026

Resumen

En este documento se describe el diseño de un generador de señales cuadradas de 100 kHz, 500 kHz, 1 MHz y 2 MHz implementado en lenguaje ensamblador para el microcontrolador ATmega328P de la tarjeta Arduino UNO. La frecuencia se selecciona mediante dos entradas digitales (PD2 y PD3) y la señal se entrega en el pin PB5, al que está conectado el LED integrado de la tarjeta. Como el reloj del sistema es de 16 MHz, cada frecuencia se obtiene conmutando el pin cada 80, 16, 8 y 4 ciclos de reloj, para lo cual se contabilizó el tiempo de ejecución de cada instrucción del programa. El código se desarrolló en Microchip Studio y su funcionamiento se verificó mediante una simulación en SimulIDE con un osciloscopio virtual.



Figura 1: Solución general del sistema.

1. Introducción

Los microcontroladores permiten generar señales digitales de frecuencia controlada mediante programa, sin necesidad de circuitos osciladores adicionales. Una forma de hacerlo es conmutar el nivel lógico de un pin de salida a intervalos regulares; cuando se programa en lenguaje ensamblador, el tiempo que tarda cada instrucción es conocido y determinista, por lo que es posible calcular con exactitud la frecuencia de la señal resultante a partir de la frecuencia del reloj del sistema [6, 2].

El presente trabajo consiste en desarrollar un programa en ensamblador para el microcontrolador ATmega328P [1] que genere una señal cuadrada en el pin PB5, al que está conectado el LED integrado de la tarjeta Arduino UNO [3]. La frecuencia de salida se selecciona con dos pines de entrada, de acuerdo con la tabla 1.

Tabla 1: Frecuencia de salida según la entrada.

PD3	PD2	Frecuencia	Ciclos por semiperiodo N
00		100 kHz	80
01		500 kHz	16
10		1 MHz	8
11		2 MHz	4

El programa se escribió y ensambló en Microchip Studio [4] y su comportamiento se comprobó mediante simulación en SimulIDE [5]. En las secciones siguientes se describen el microprocesador, las memorias y la interfaz de periféricos utilizados, el algoritmo del programa, el análisis de los resultados obtenidos y las conclusiones.

frecuencia [1]. Utiliza una arquitectura Harvard, con memorias y buses separados para programa y datos, y un *pipeline* de dos etapas: mientras se ejecuta una instrucción, la siguiente se obtiene de la memoria de programa. Puede operar hasta 20 MHz; la tarjeta Arduino UNO emplea un cristal de 16 MHz [3], de modo que el periodo del reloj es

$$T_{clk} = \frac{1}{f_{clk}} = \frac{1}{16 \text{ MHz}} = 62,5 \text{ ns.} \quad (1)$$

El procesador cuenta con 32 registros de propósito general de 8 bits (R0–R31) conectados directamente a la unidad aritmético-lógica, lo que permite acceder a dos registros en una sola instrucción. Los registros R26 a R31 pueden agruparse en pares para formar los apuntadores X, Y y Z. El registro de estado SREG almacena las banderas I, T, H, S, V, N, Z y C; en este programa se utiliza la bandera de cero (Z), que modifican las instrucciones CPI y DEC y consultan las instrucciones de salto condicional [1, 2]. La tabla 2 resume los registros empleados.

Tabla 2: Registros utilizados en el programa.

Registro	Uso
r16	Máscara 0x20 (bit de PB5); también configura DDRB
r17	Lectura y enmascaramiento de PIND
r18	Contador de los bucles de retardo
DDRB (0x04)	Dirección de PORTB: PB5 como salida
DDRD (0x0A)	Dirección de PORTD: PD2 y PD3 como entradas
PIND (0x09)	Lectura de las entradas PD2 y PD3
PINB (0x03)	Escribir un 1 en PB5 conmuta su nivel
SREG	Bandera Z para las comparaciones y saltos

Las instrucciones LDI, ANDI y CPI solo operan sobre los registros R16 a R31, por lo que los registros de trabajo del programa se eligieron dentro de ese in-

Tabla 3: Ciclos de reloj por instrucción.

Instrucción	Operación	Ciclos
OUT	$E/S \leftarrow Rr$	1
IN	$Rd \leftarrow E/S$	1
LDI	$Rd \leftarrow K$	1
ANDI	$Rd \leftarrow Rd \wedge K$	1
CPI	Compara Rd con K	1
DEC	$Rd \leftarrow Rd - 1$	1
NOP	Sin operación	1
BREQ, BRNE	Salto condicional	1 / 2*
RJMP	Salto relativo	2

* 1 ciclo si no se cumple la condición, 2 si se cumple.

borrado. Aquí se almacena el programa en ensamblador una vez ensamblado y cargado (archivo `.hex`); el programa ocupa tan solo unas decenas de palabras, menos del 1 % de su capacidad.

- **SRAM:** memoria de datos volátil de 2 kB. Dentro del espacio de datos se ubican, antes de la SRAM, los 32 registros de trabajo, los 64 registros de E/S y los 160 registros de E/S extendidos; la SRAM interna ocupa las direcciones 0x0100 a 0x08FF. En este programa no se emplea SRAM ni pila, pues no hay subrutinas ni variables en memoria.
- **EEPROM:** memoria no volátil de 1 kB con un espacio de direcciones independiente, accesible mediante los registros `EEAR`, `EEDR` y `EECR`, con una resistencia de al menos 100 000 ciclos de escritura y borrado. No se utiliza en este programa.

2.3. Interfaz para periféricos

El microcontrolador se comunica con el exterior mediante puertos de E/S de 8 bits bidireccionales (`PORTB`, `PORTC` y `PORTD`). Cada pin se controla con tres registros: `DDRx`, que define su dirección (1 para salida, 0 para entrada); `PORTx`, que establece el nivel de salida o activa la resistencia de pull-up interna cuando el pin es entrada; y `PINx`, que permite leer el nivel presente en el pin. Además, escribir un 1 en un bit de `PINx` conmuta el valor del bit correspondiente de `PORTx`, independientemente de `DDRx`; esta propiedad permite cambiar el nivel de PB5 con una sola instrucción `OUT` de un ciclo [1]. La lectura de un pin pasa por un sincronizador que introduce un retardo de entre 0.5 y 1.5 ciclos de reloj.

Los pines utilizados se muestran en la tabla 4.

Las características eléctricas de los puertos para una alimentación de 5 V se resumen en la tabla 5 [1]. Las entradas se conectan directamente a 0 V (nivel 0) o a 5 V (nivel 1), valores que están dentro de los umbrales de entrada. El pin PB5 alimenta al LED a través de una resistencia de 220 Ω ; suponiendo una

Tabla 4: Pines utilizados.

Pin	Arduino	Tipo	Función
PD2	D2	Entrada	Bit 0 de selección
PD3	D3	Entrada	Bit 1 de selección
PB5	D13	Salida	Señal / LED integrado

caída de aproximadamente 2 V en el LED, la corriente es del orden de 14 mA, inferior a los 20 mA para los que están especificados los niveles de salida.

Tabla 5: Características eléctricas de los puertos ($V_{CC} = 5$ V).

Parámetro	Valor
Tensión de operación (hasta 20 MHz)	4,5 V a 5,5 V
Nivel bajo de entrada V_{IL}	máx. 0,3 V_{CC} (1,5 V)
Nivel alto de entrada V_{IH}	mín. 0,6 V_{CC} (3,0 V)
Nivel bajo de salida V_{OL}	máx. 0,9 V ($I_{OL} = 20$ mA)
Nivel alto de salida V_{OH}	mín. 4,2 V ($I_{OH} = -20$ mA)
Resistencia de pull-up interna	20 k Ω a 50 k Ω
Corriente máx. por pin (DC)	40 mA (límite absoluto)
Corriente de fuga de entrada	máx. 1 μ A

3. Algoritmo del programa

3.1. Cálculo de la temporización

Para obtener una señal cuadrada de frecuencia f con un ciclo de trabajo de 50 %, el pin debe conmutar dos veces por periodo, es decir, cada semiperiodo. Si N es el número de ciclos de reloj que dura un semiperiodo, entonces

$$f = \frac{f_{clk}}{2N} \iff N = \frac{f_{clk}}{2f}. \quad (2)$$

Con $f_{clk} = 16$ MHz se obtienen los valores de N de la tabla 1: 80 ciclos (5 μ s) para 100 kHz, 16 para 500 kHz, 8 para 1 MHz y 4 para 2 MHz.

Los retardos se construyen con un contador que se decremanta en un bucle (`LDI`, `DEC`, `BRNE`). Un bucle con valor inicial K consume exactamente $3K$ ciclos: `LDI` (1) más $K - 1$ vueltas de `DEC` y `BRNE` tomado (3 ciclos cada una) más una última vuelta con el salto no tomado (2 ciclos). Los retardos restantes se completan con instrucciones `NOP` [6].

Cada rutina conmuta PB5 con `OUT PINB`, `r16`, espera, vuelve a conmutar, espera y, antes de repetir, lee las entradas (`IN`, `ANDI`, `CPI`, 3 ciclos) y salta (`BREQ` tomado, 2 ciclos) si la selección no cambió. Esas instrucciones forman parte del semiperiodo bajo, por lo que el retardo de esa fase es menor. Para 2 MHz, donde $N = 4$, la lectura de las entradas se coloca dentro del semiperiodo alto. La tabla 6 resume los retardos utilizados.

Tabla 6: Retardos de cada rutina (en ciclos).

f	N	Fase alta	Fase baja
100 kHz	80	$K=26 + 1 \text{ NOP}$ (79)	$K=24 + 2 \text{ NOP}$ (74)
500 kHz	16	$K=5$ (15)	$K=3 + 1 \text{ NOP}$ (10)
1 MHz	8	$K=2 + 1 \text{ NOP}$ (7)	2 NOP (2)
2 MHz	4	IN, ANDI, CPI (3)	1 NOP (1)

3.2. Pseudocódigo

Los algoritmos 1, 2 y 3 describen el programa completo con la misma estructura que el código en ensamblador del código 1. El algoritmo 1 corresponde a las etiquetas RESET y LEER; el algoritmo 2, a las rutinas F100K, F500K y F1M, que solo difieren en sus retardos R_a y R_b (tabla 6); y el algoritmo 3, a la rutina F2M.

Algoritmo 1: Funcionamiento del programa completo

Datos: $f_{clk} = 16 \text{ MHz}$, máscara de PB5
 $0x20$, máscara de entradas $0x0C$
Entrada: Bits de selección $S = (\text{PD3}, \text{PD2})$
Salida: Señal cuadrada en PB5
Resultado: Onda cuadrada de 100 kHz,
500 kHz, 1 MHz o 2 MHz según
 S

```

1 inicio
2   r16 ← 0x20
3   DDRB ← r16
4   DDRD ← 0x00
5   repetir
6     S ← PIND ∧ 0x0C
7     si S = 0x00 entonces
8       Ejecutar rutina de 100 kHz
        (algoritmo 2) con  $R_a = 79$  y
         $R_b = 74$ 
9     si no, si S = 0x04 entonces
10      Ejecutar rutina de 500 kHz
        (algoritmo 2) con  $R_a = 15$  y
         $R_b = 10$ 
11     si no, si S = 0x08 entonces
12      Ejecutar rutina de 1 MHz
        (algoritmo 2) con  $R_a = 7$  y
         $R_b = 2$ 
13     si no
14      Ejecutar rutina de 2 MHz
        (algoritmo 3)
15 hasta que se retire la alimentación

```

Algoritmo 2: Rutina de 100 kHz, 500 kHz y 1 MHz para la selección S_i

Datos: Selección S_i , retardos R_a y R_b (en ciclos), $r16 = 0x20$

Resultado: Onda cuadrada con ciclo de trabajo de 50 % mientras $S = S_i$

```

1 inicio
2   repetir
3     PINB ← r16 (conmutar PB5)
4     Retardo de  $R_a$  ciclos
5     PINB ← r16 (conmutar PB5)
6     Retardo de  $R_b$  ciclos
7     S ← PIND ∧ 0x0C
8   hasta que S ≠ Si
9   Regresar a la lectura de S (LEER)

```

Algoritmo 3: Rutina de 2 MHz

Datos: $r16 = 0x20$, $S_i = 0x0C$

Resultado: Onda cuadrada de 2 MHz con ciclo de trabajo de 50 % mientras $S = S_i$

```

1 inicio
2   repetir
3     PINB ← r16 (conmutar PB5)
4     S ← PIND ∧ 0x0C
5     PINB ← r16 (conmutar PB5)
6     Retardo de 1 ciclo (NOP)
7   hasta que S ≠ Si
8   Regresar a la lectura de S (LEER)

```

3.3. Código en ensamblador

El código 1 muestra el programa completo, desarrollado en Microchip Studio, que implementa los algoritmos anteriores. Los comentarios indican el número de ciclos de cada retardo, de acuerdo con la tabla 6.

Código 1: Programa completo en ensamblador (`programa_act1.asm`).

```

1 ;
2 ; programa_act1.asm
3 ;
4 ; Created: 28/09/2026 10:28:47 p. m.
5 ; Author : alfonso
6 ;
7 .include "m328Pdef.inc"
8
9 ;----- configuracion de puertos -----
10 RESET:
11   ldi r16, (1<<PB5)      ; mascara de PB5 (se conserva
    ↪ en r16)
12   out DDRB, r16          ; PB5 como salida
13   ldi r17, 0x00
14   out DDRD, r17          ; PORTD como entrada (PD2 y
    ↪ PD3)
15
16 ;----- lectura y seleccion -----
17 LEER:
18   in r17, PIND
19   andi r17, 0x0C          ; aislar PD3 y PD2
20   cpi r17, 0x00
21   breq F100K
22   cpi r17, 0x04
23   breq F500K
24   cpi r17, 0x08

```

```

25     breq F1M
26     rjmp F2M                ; 0x0C
27
28 ;----- 100 kHz (medio periodo = 80 ciclos) --
29 F100K:
30     out PINB, r16            ; conmuta PB5 (alto)
31     ldi r18, 26              ; 78 ciclos
32 D1: dec r18
33     brne D1
34     nop                      ; +1 -> 79 de retardo
35     out PINB, r16            ; conmuta PB5 (bajo)
36     ldi r18, 24              ; 72 ciclos
37 D2: dec r18
38     brne D2
39     nop
40     nop                      ; +2 -> 74 de retardo
41     in r17, PIND
42     andi r17, 0x0C
43     cpi r17, 0x00
44     breq F100K              ; sigue igual -> repetir
45     rjmp LEER               ; cambio -> volver a leer
46
47 ;----- Modulo 4: 500 kHz (medio periodo = 16 ciclos)
48     ↪ --
49 F500K:
50     out PINB, r16
51     ldi r18, 5              ; 15 ciclos
52 D3: dec r18
53     brne D3
54     out PINB, r16
55     ldi r18, 3              ; 9 ciclos
56 D4: dec r18
57     brne D4
58     nop                      ; +1 -> 10
59     in r17, PIND
60     andi r17, 0x0C
61     cpi r17, 0x04
62     breq F500K
63     rjmp LEER
64 ;----- 1 MHz (medio periodo = 8 ciclos) -----
65 F1M:
66     out PINB, r16
67     ldi r18, 2              ; 6 ciclos
68 D5: dec r18
69     brne D5
70     nop                      ; +1 -> 7
71     out PINB, r16
72     nop
73     nop                      ; 2 de retardo
74     in r17, PIND
75     andi r17, 0x0C
76     cpi r17, 0x08
77     breq F1M
78     rjmp LEER
79
80 ;----- Modulo 6: 2 MHz (medio periodo = 4 ciclos)
81     ↪ -----
82 F2M:
83     out PINB, r16            ; t=0
84     in r17, PIND            ; t=1
85     andi r17, 0x0C          ; t=2
86     cpi r17, 0x0C          ; t=3
87     out PINB, r16            ; t=4
88     nop                      ; t=5
89     breq F2M                ; t=6-7 (tomado = 2 ciclos) ->
90     ↪ periodo 8
91     rjmp LEER

```

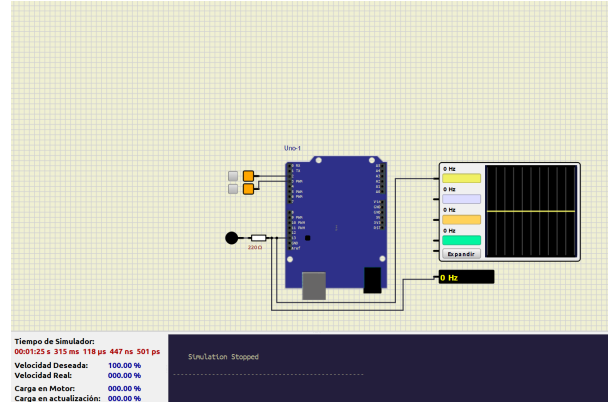


Figura 2: Circuito de prueba en SimulIDE.

osciloscopio para cada combinación de las entradas. En cada caso se observa una señal cuadrada en PB5 con un ciclo de trabajo de 50 %, y el periodo medido en pantalla se compara con el teórico en la tabla 7. Las diferencias entre ambos son atribuibles a la resolución de lectura del osciloscopio virtual, pues la temporización del programa está determinada por el número exacto de ciclos de cada instrucción.

Tabla 7: Frecuencia teórica y medida.

S	Teórica	Periodo teórico	Periodo medido
00	100 kHz	10 µs	COMPLETAR
01	500 kHz	2 µs	COMPLETAR
10	1 MHz	1 µs	COMPLETAR
11	2 MHz	500 ns	COMPLETAR

Al cambiar la combinación de las entradas con la simulación en marcha, el programa sale de la rutina en ejecución y vuelve a leer la selección, por lo que la nueva frecuencia aparece a partir del siguiente periodo. Respecto al LED, a estas frecuencias el ojo humano no percibe el parpadeo: el LED se aprecia encendido de forma continua, con una luminosidad aparente de aproximadamente la mitad de la nominal por el ciclo de trabajo de 50 %. Por ello, la verificación de la frecuencia requiere un osciloscopio o un frecuencímetro. Además, en una tarjeta física hay que considerar que a 2 MHz los tiempos de subida y bajada de la señal, determinados por la capacitancia del pin y de las conexiones, se vuelven apreciables frente al semiperiodo de 250 ns.

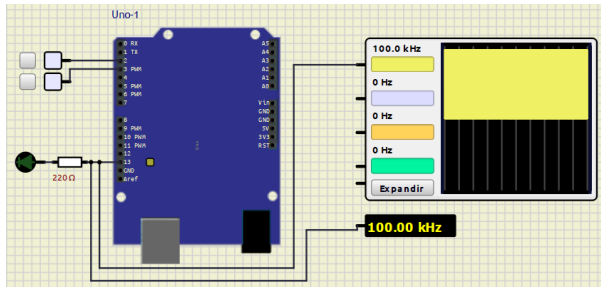
4. Análisis de resultados

El programa se ensambló en Microchip Studio sin errores y el archivo .hex generado se cargó en el Arduino UNO del simulador SimulIDE. El circuito de prueba (figura 2) consta de dos fuentes de voltaje fijo conectadas a D2 y D3 para elegir la frecuencia, el LED con su resistencia de $220\ \Omega$ en D13 y un osciloscopio con el canal A conectado a D13.

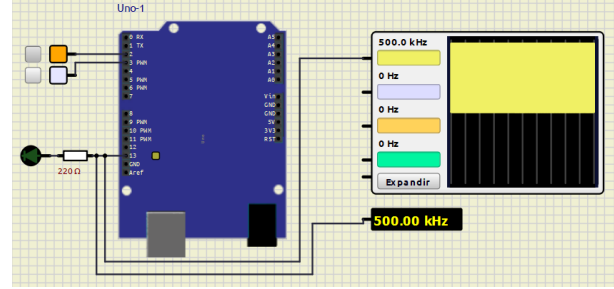
En la figura 3 se muestran las capturas del os-

5. Conclusión

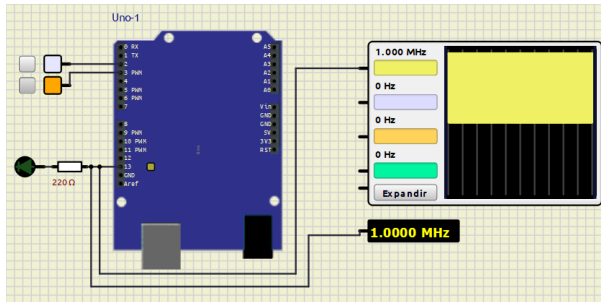
Se desarrolló en lenguaje ensamblador un generador de señales cuadradas de cuatro frecuencias seleccionables (100 kHz, 500 kHz, 1 MHz y 2 MHz) para el ATmega328P. Se comprobó que, al programar en ensamblador, el conocimiento del número de ciclos que consume cada instrucción permite controlar con



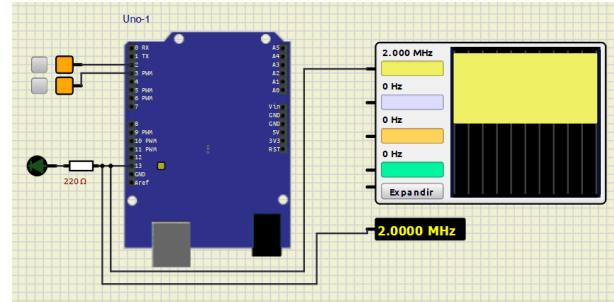
(a) $S = 00$: 100 kHz.



(b) $S = 01$: 500 kHz.



(c) $S = 10$: 1 MHz.



(d) $S = 11$: 2 MHz.

Figura 3: Señal en PB5 para cada combinación de las entradas.

precisión el tiempo y, con él, la frecuencia de la señal, sin recurrir a temporizadores. El uso del registro `PINx` para conmutar un pin con una sola instrucción fue clave para alcanzar los 2 MHz, ya que con ello el semiperiodo de solo 4 ciclos alcanza para incluir también la lectura de las entradas.

También se observó la importancia de conocer la arquitectura del microcontrolador (registros, puertos y sus características eléctricas) para seleccionar adecuadamente los pines y los valores de los componentes externos. Finalmente, la simulación en SimulIDE permitió validar el diseño antes de probarlo en la tarjeta Arduino UNO.

Referencias

- [1] Microchip Technology Inc. “ATmega48A/PA/88A/PA/168A/PA/328/P Data Sheet, DS40002061.” (s. f.), dirección: <https://www.microchip.com/en-us/product/ATmega328P> (visitado 30-09-2026).
- [2] Microchip Technology Inc. “AVR Instruction Set Manual, DS40002198.” (s. f.), dirección: <https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-Instruction-Set-Manual-DS40002198.pdf> (visitado 30-09-2026).
- [3] Arduino. “Arduino UNO Rev3.” (s. f.), dirección: <https://docs.arduino.cc/hardware/uno-rev3/> (visitado 30-09-2026).
- [4] Microchip Technology Inc. “Microchip Studio for AVR and SAM devices.” (s. f.), dirección: <https://www.microchip.com/en-us/>

[tools-resources/develop/microchip-studio](https://www.microchip.com/en-us/tools-resources/develop/microchip-studio) (visitado 30-09-2026).

- [5] SimulIDE. “Real Time Electronic Circuit Simulator.” (s. f.), dirección: <https://simulide.com> (visitado 30-09-2026).
- [6] M. A. Mazidi, S. Naimi y S. Naimi, *The AVR Microcontroller and Embedded Systems: Using Assembly and C*. Pearson, 2011.